Labo 3: Geavanceerde Computerarchitectuur

Gust Brouckaert
Tim Depickere

15 november 2024

1 Introductie

In dit derde labo verdiepten we ons in geavanceerde concepten zoals **warps**, **striding** en **thread divergence** binnen CUDA-programmering. De opdracht bestond uit het inverteren van de kleuren van een afbeelding, waarbij de focus lag op het analyseren van de impact van thread divergence en warp-efficiëntie. Daarnaast onderzochten we de prestatieverschillen bij het gebruik van verschillende configuraties voor *blocks* en *threads*.

2 Opdrachtbeschrijving en Implementatie

In dit labo moesten we een afbeelding manipuleren door de RGB-waarden van de pixels aan te passen. De opdracht bestond uit twee delen:

- **Deel 1:** Het inverteren van alle RGB-componenten van de afbeelding.
- **Deel 2:** Specifieke bewerkingen uitvoeren op de rode component (`R`) wanneer deze binnen een bepaald bereik valt, en de overige componenten inverteren.

Onze implementatie maakt gebruik van een striding-mechanisme, waarmee threads op efficiënte wijze pixels in de afbeelding verwerken. De kern van onze implementatie is de volgende kernel-functie:

Listing 1: CUDA-kernel voor het verwerken van een afbeelding

```
__global__ void invertPhoto(uint8_t* array, int blocks) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int max = M * N * C;  //Totaal aantal elementen in de afbeelding
    int stride = blockDim.x * gridDim.x;

    for (int i = idx; i < max; i += stride) {
        if (i % 3 == 0 && array[i] >= 100 && array[i] <= 200) {
            array[i] = (array[i] % 25) * 10;
        } else {
            array[i] = 255 - array[i];
        }
    }
}
```

De kernel voert de volgende stappen uit:

1. Elk element in de array wordt geïdentificeerd aan de hand van de globale index `idx`, gebaseerd op de thread- en block-configuratie.

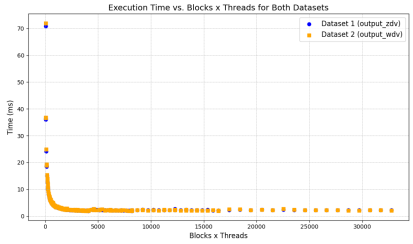
2. Het `stride`-mechanisme zorgt ervoor dat threads werken op niet-overlappende subsets van de array.

3. Voor de rode component (`R`) worden specifieke bewerkingen uitgevoerd indien de waarde binnen het bereik [100, 200] valt. Voor alle andere componenten wordt de waarde geïnverteerd.

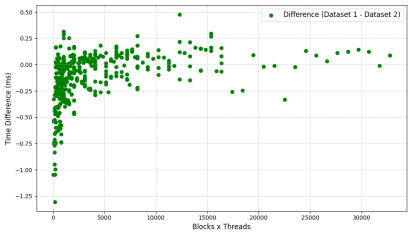
3 Prestaties en Analyse

Om de impact van thread divergence te evalueren, hebben we tests uitgevoerd met verschillende combinaties van *blocks* en *threads*. We maten de uitvoertijd voor scenario's met en zonder thread divergence. De resultaten zijn weergegeven in onderstaande grafieken.

3.1 Resultaten



(a) Runtimes: zonder thread divergence vs. met thread divergence



(b) Tijdsverschil: zonder divergence - met divergence

Figure 1: Vergelijking van prestaties bij verschillende configuraties van threads en blocks.

- **Impact van thread divergence:** Uit figuur 1a blijkt dat scenario's zonder thread divergence consistent sneller zijn. Dit verschil wordt vooral merkbaar bij een lagere configuratie van threads en blocks, zoals zichtbaar in figuur 1b.

- **Prestatieverschil:** Het maximale tijdsverschil bedraagt ongeveer 1,25 ms bij configuraties met weinig threads en blocks. In verdere scenario's zitten we met een runtime van rond de 2ms en is het verschil ongeveer 0.2ms. Dit is 10% sneller wat al redelijk iets is.